

Министерство образования и науки РФ

Федеральное государственное автономное образовательное учебное
учреждение высшего образования “Санкт-Петербургский
национальный исследовательский университет ИТМО”

Факультет прикладной информатики

Лабораторная работа №3

По дисциплине: Проектирование и реализация баз данных

Выполнил:

Ситдинов Рафаэль Ильдусович

Группа: К3240

Проверила:

Говорова Марина Михайловна

Г. Санкт-Петербург, 2026г.

Задание:

Вариант 14. БД «Служба заказа такси»

Описание предметной области: Система должна фиксировать все вызовы такси и распределять их между водителями.

Каждому водителю ежедневно начисляется заработная плата в зависимости от количества вызовов и их тарифа (50% от заработанной им суммы). Автомобили могут быть собственностью компании или таксиста.

Заказ принимает дежурный администратор и передает его водителю. В заказе фиксируется тип оплаты – наличными или онлайн. Если заказ оплачивается онлайн, то в системе хранится привязка к карте, с которой была совершена оплата.

В системе необходимо хранить график работы водителей.

Ежедневно действуют базовые тарифы на тип предоставляемых авто, но в зависимости от времени суток и ситуации на дорогах, цена может корректироваться.

БД должна содержать следующий минимальный набор сведений: Код сотрудника. ФИО сотрудника. Адрес сотрудника. № телефона сотрудника. Паспортные данные сотрудника. Должность сотрудника. Категория сотрудника. Наименование модели и марки автомобиля. Технические характеристики. Стран-производитель. Стоимость. Код тарифа. Наименование тарифа. Цена за километр. Код автомобиля. Госномер автомобиля. Год выпуска. Пробег. Дата последнего ТО. Дата вызова. Время посадки пассажира. Время высадки пассажира. Номер телефона пассажира. Откуда. Куда. Расстояние. Штраф за время ожидания (в минутах). Оплата (онлайн (при заказе) или наличными). Рекламация клиента на вызов.

Дополните состав атрибутов на основе анализа предметной области.

Описание предметной области:

Система предназначена для фиксации всех вызовов такси и распределения их между водителями.

Заказ принимает водитель. Для каждого заказа фиксируются сведения о клиенте, времени выполнения, типе оплаты и итоговой стоимости поездки.

Каждому водителю ежедневно начисляется заработная плата в зависимости от количества выполненных вызовов и их тарифа. Автомобили могут принадлежать компании или водителю (статус принадлежности хранится в системе).

В заказе фиксируется тип оплаты – наличными или онлайн. Если заказ оплачивается онлайн, то в системе хранится привязка к банковской карте, с которой была совершена оплата.

В системе хранится график работы сотрудников (включая дни работы, отгулы, отпуск, больничный и информацию о свободном/выставленном графике).

Ежедневно действуют базовые тарифы на тип предоставляемых авто (эконом/комфорт/бизнес), но в зависимости от времени суток и ситуации на дорогах цена может корректироваться.

Минимальный набор сведений, хранимый в БД, включает:

- данные сотрудников (код, ФИО, телефон, паспортные данные, должность, категория);
- данные клиентов (код клиента, имя, рейтинг);
- данные автомобилей и моделей (код автомобиля, госномер, год выпуска, технические характеристики, страна-производитель);
- данные тарифов (код, наименование, цена за километр, корректировки);
- данные заказов (дата вызова, время посадки/высадки, расстояние, стоимость, тип оплаты, рекламации).

Инфологическая модель:

Для проектирования БД выделены основные сущности предметной области:

- Клиент – пользователь сервиса такси, оформляющий заказ.
- Заказ – факт вызова такси, содержащий данные поездки и оплаты.
- Сотрудник – работник сервиса такси.
- Договор – документ, описывающий условия работы сотрудника.
- Должность – справочник должностей сотрудников.
- График работы – расписание работы сотрудника.
- Автомобиль – юридическая единица авто, участвующая в заказах.
- Модель – описание марки/модели авто и технических характеристик.
- Тариф – тарифные правила для расчёта стоимости поездки.
- Банковская карта – платёжный инструмент для онлайн-оплаты.

В результате анализа предметной области построена ER-диаграмма, содержащая атрибуты и связи между сущностями.

Диаграмма:

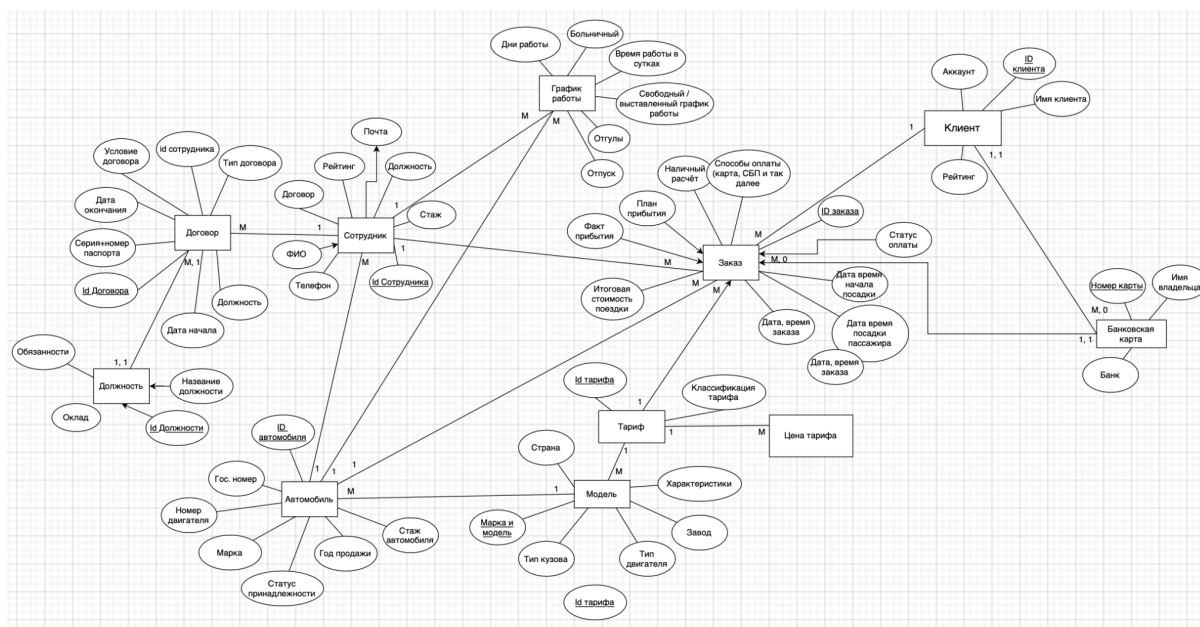
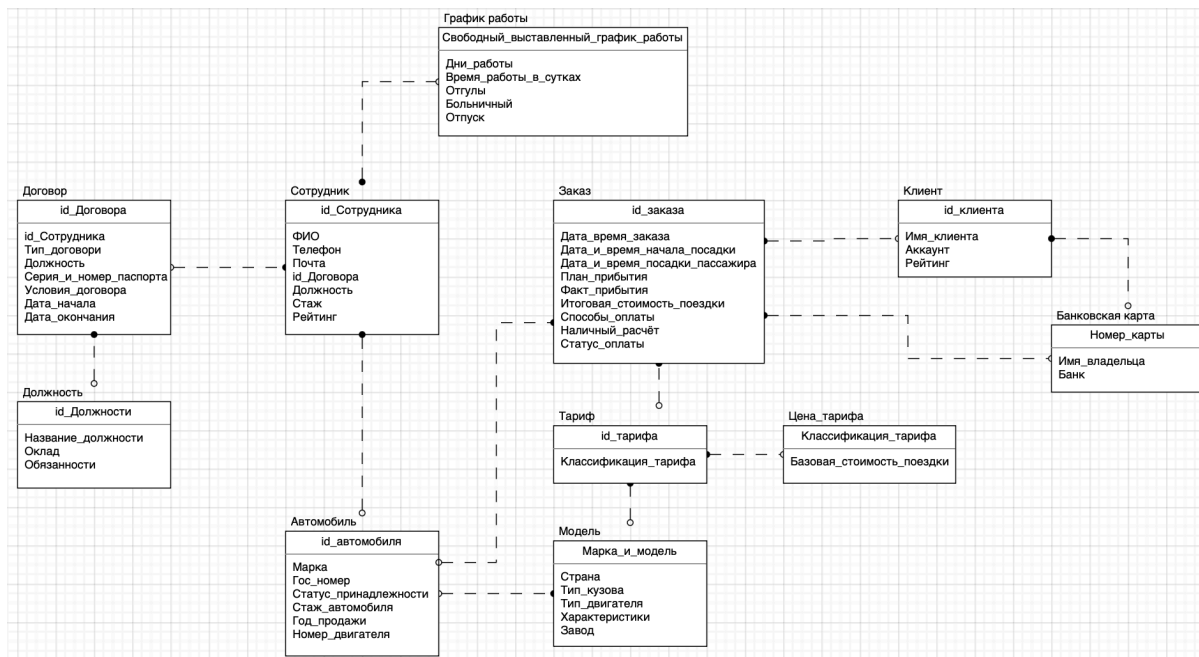


Схема инфологической модели данных БД в нотации IDEF1X:



Сущности:

1. Клиент

Атрибут	Тип	PK	FK	Обязательность	Ограничения
id_клиента	INTEGER	+	—	+	Уникален, 10 символов, авто-генерация
Имя_Клиента	STRING	—	—	+	Только буквы (3-80)
Аккаунт	STRING	—	—	+	Уникален
Рейтинг	FLOAT	—	—	—	0.0 – 5.0

2. Банковская карта

Атрибут	Тип	PK	FK	Обязательность	Ограничения
Номер_карты	INTEGER	+	—	+	Уникален, 16 цифр
Имя_Владельца	STRING	—	—	+	Только буквы, через пробел, два слова
Банк	STRING	—	—	+	не NULL

3. Сотрудник

Атрибут	Тип	PK	FK	Обязательность	Ограничения
ID_сотрудника	INTEGER	+	–	+	Уникален, 7 символов, авто-генерация
ФИО	STRING	–	–	+	Только буквы, через пробел, три слова
Телефон	STRING	–	–	+	Уникален, формат +7...
Почта	STRING	–	–	+	Уникален
id_Договора	STRING	–	+	+	из “Договор”
Должность	STRING	–	–	+	Из шаблона
Стаж	FLOAT	–	–	+	18.0 – 80.0 лет
Рейтинг	FLOAT	–	–	+	0.0 – 5.0

4. Договор

Атрибут	Тип	PK	FK	Обязательность	Ограничения
ID_договора	INTEGER	+	–	+	Уникален, 7 символов, авто-генерация
Тип_договора	STRING	–	–	+	Из шаблона
Должность	STRING	–	+	+	из “Должность”
Паспортные_данные	INTEGER	–	–	+	Уникален
Условия_договора	STRING	–	–	+	Не NULL
Дата_начала	DATE	–	–	+	Дата начала < дата окончания, не NULL
Дата_окончания	DATE	–	–	+	Дата начала < дата окончания, не NULL

5. Должность

Атрибут	Тип	PK	FK	Обязательность	Ограничения
id_должности	INTEGER	+	–	+	Уникален, 4 символов, авто-генерация
Название_должности	STRING	–	–	+	Из шаблона

Оклад	INTEGER	–	–	+	не NULL, больше 0
Обязанности	STRING	–	–	+	не NULL

6. Автомобиль

Атрибут	Тип	PK	FK	Обязательность	Ограничения
ID_автомобиля	INTEGER	+	–	+	Уникален, 10 символов, авто-генерация
Марка	STRING	–	+	+	из “Марка_модель”
Гос_номер	STRING	–	+	+	Уникален, формат гос-номера
Статус_принадлежности	STRING	–	–	+	“Да” или “Нет”
Стаж_автомобиля	FLOAT	–	–	+	Не NULL, больше 0.0
Год_продажи	DATE	–	–	+	Формат (00.00.0000), год > 1990
Номер_двигателя	DATE	–	–	+	не NULL

7. Марка и модель

Атрибут	Тип	PK	FK	Обязательность	Ограничения
Марка_модель	STRING	+	–	+	не NULL
Страна	STRING	–	–	+	из списка стран
Тип_кузова	STRING	–	–	+	из шаблона
Тип_двигателя	STRING	–	–	+	из шаблона
Характеристики	LIST	–	–	+	
Завод	STRING	–	–	+	не NULL

8. Тариф

Атрибут	Тип	PK	FK	Обязательность	Ограничения
id_тарифа	INTEGER	+	–	+	Уникален, 3 цифры
Классификация	STRING	–	+	+	из “Цена тарифа”

9. Цена тарифа

Атрибут	Тип	PK	FK	Обязательность	Ограничения
Классификация	STRING	+	–	+	из шаблона
базовая_стоимость	INTEGER	–	–	+	из шаблона

10. Заказ

Атрибут	Тип	PK	FK	Обязательность	Ограничения
ID_заказа	INTEGER	+	–	+	Уникален, 30 символов, авто-генерация
Дата_время_заказа	DATA	–	+	+	не NULL
ДВ_начала_посадки	TIMESTAMP	–	+	+	ДВ_начала_посадки < ДВ_посадки_пассажира
ДВ_посадки_пассажира	TIMESTAMP	–	–	+	ДВ_начала_посадки < ДВ_посадки_пассажира
План_прибытия	DATE	–	–	+	Время, назначенное к прибытию
Факт_прибытия	DATE	–	–	+	Фактическое время прибытия
Итоговая_стоимость_поездки	INTEGER	–	–	+	Формат (00.00.0000), год > 1990
Способ_оплаты	STRING	–	–	+	не NULL, карта, СБП, QR, Нет
Наличный_расчёт	STRING	–	–	+	Да: Если “Способ_оплаты” == “Нет”. Нет: Если иначе
Статус_оплаты	STRING	–	–	+	Да: Если оплата произошла Нет: Иначе

Код для PostgreSQL:

Создание схемы:

```
CREATE SCHEMA taxi;
```

1. Должность:

```
CREATE TABLE taxi.position (  
  id_position BIGSERIAL PRIMARY KEY,  
  position_name VARCHAR(100) NOT NULL,  
  salary INTEGER NOT NULL,  
  duties TEXT NOT NULL,  
  
  CONSTRAINT chk_salary_positive  
    CHECK (salary > 0));
```

2. Договор:

```
CREATE TABLE taxi.contract (  
  id_contract BIGSERIAL PRIMARY KEY,  
  contract_type VARCHAR(50) NOT NULL,  
  position_id BIGINT NOT NULL,  
  passport_data VARCHAR(20) NOT NULL,  
  contract_terms TEXT NOT NULL,  
  start_date DATE NOT NULL,  
  end_date DATE NOT NULL,  
  
  CONSTRAINT fk_contract_position  
    FOREIGN KEY (position_id)  
    REFERENCES taxi.position(id_position));
```

3. Клиенты:

```
CREATE TABLE taxi.client (  
  id_client BIGSERIAL PRIMARY KEY,  
  client_name VARCHAR(80) NOT NULL,  
  account VARCHAR(100) UNIQUE NOT NULL,  
  rating NUMERIC(2,1),  
  
  CONSTRAINT chk_client_rating  
    CHECK (rating >= 0 AND rating <= 5));
```

4. Банковские карты:

```
CREATE TABLE taxi.bank_card (  
  card_number VARCHAR(16) PRIMARY KEY,  
  owner_name VARCHAR(100) NOT NULL,  
  bank VARCHAR(100) NOT NULL  
);
```

5. Модели автомобилей:

```
CREATE TABLE taxi.car_model (  
  model_name VARCHAR(100) PRIMARY KEY,  
  country VARCHAR(50) NOT NULL,  
  body_type VARCHAR(50) NOT NULL,  
  engine_type VARCHAR(50) NOT NULL,  
  characteristics TEXT,  
  factory VARCHAR(100) NOT NULL  
);
```

6. Автомобили:

```
CREATE TABLE taxi.car (  
  id_car BIGSERIAL PRIMARY KEY,  
  model_name VARCHAR(100) NOT NULL,  
  ownership_status VARCHAR(20) NOT NULL,  
  car_age NUMERIC(4,1) NOT NULL,  
  production_year INTEGER NOT NULL,  
  engine_number VARCHAR(50) NOT NULL,  
  
  CONSTRAINT fk_car_model  
    FOREIGN KEY (model_name)  
    REFERENCES taxi.car_model(model_name));
```

7. Стоимости тарифов:

```
CREATE TABLE taxi.tariff_price (  
  tariff_class VARCHAR(30) PRIMARY KEY,  
  base_price INTEGER NOT NULL,  
  
  CONSTRAINT chk_tariff_price_positive  
    CHECK (base_price > 0)  
);
```

8. Сотрудники:

```
CREATE TABLE taxi.employee (  
  id_employee BIGSERIAL PRIMARY KEY,  
  full_name VARCHAR(120) NOT NULL,  
  phone VARCHAR(20) UNIQUE NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  contract_id BIGINT NOT NULL,  
  experience NUMERIC(4,1) NOT NULL,  
  rating NUMERIC(2,1) NOT NULL,  
  
  CONSTRAINT fk_employee_contract  
    FOREIGN KEY (contract_id)  
    REFERENCES taxi.contract(id_contract),  
  
  CONSTRAINT chk_employee_experience  
    CHECK (experience >= 0),  
  
  CONSTRAINT chk_employee_rating  
    CHECK (rating >= 0 AND rating <= 5)  
);
```

9. Заказы:

```
CREATE TABLE taxi.orders (  
  id_order BIGSERIAL PRIMARY KEY,  
  
  client_id BIGINT NOT NULL,  
  employee_id BIGINT NOT NULL,  
  car_id BIGINT NOT NULL,  
  tariff_id BIGINT NOT NULL,  
  
  card_number VARCHAR(16),  
  
  order_datetime TIMESTAMP NOT NULL,  
  boarding_start TIMESTAMP NOT NULL,  
  passenger_boarding TIMESTAMP NOT NULL,  
  plannet_arrival TIMESTAMP NOT NULL,  
  actual_arrival TIMESTAMP NOT NULL,  
  
  trip_cost INTEGER NOT NULL,
```

```
payment_method VARCHAR(20) NOT NULL,  
cash_payment VARCHAR(3) NOT NULL,  
payment_status VARCHAR(3) NOT NULL,
```

```
complaint TEXT,
```

```
CONSTRAINT fk_orders_client  
    FOREIGN KEY (client_id)  
    REFERENCES taxi.client(id_client),
```

```
CONSTRAINT fk_orders_employee  
    FOREIGN KEY (employee_id)  
    REFERENCES taxi.employee(id_employee),
```

```
CONSTRAINT fk_orders_car  
    FOREIGN KEY (car_id)  
    REFERENCES taxi.car(id_car),
```

```
CONSTRAINT fk_orders_tariff  
    FOREIGN KEY (tariff_id)  
    REFERENCES taxi.tariff(id_tariff),
```

```
CONSTRAINT fk_orders_bank_card  
    FOREIGN KEY (card_number)  
    REFERENCES taxi.bank_card(card_number),
```

```
CONSTRAINT chk_trip_cost  
    CHECK (trip_cost >= 0)
```

```
);
```

Тестирование данных в таблицы

1. Проверка списка таблиц:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'taxi'
ORDER BY table_name;
```

2. Проверка количества записей:

```
SELECT 'position' AS table_name, COUNT(*) FROM taxi.position
UNION ALL
SELECT 'tariff_price', COUNT(*) FROM taxi.tariff_price
UNION ALL
SELECT 'car_model', COUNT(*) FROM taxi.car_model
UNION ALL
SELECT 'client', COUNT(*) FROM taxi.client
UNION ALL
SELECT 'bank_card', COUNT(*) FROM taxi.bank_card
UNION ALL
SELECT 'contract', COUNT(*) FROM taxi.contract
UNION ALL
SELECT 'car', COUNT(*) FROM taxi.car
UNION ALL
SELECT 'tariff', COUNT(*) FROM taxi.tariff
UNION ALL
SELECT 'employee', COUNT(*) FROM taxi.employee
UNION ALL
SELECT 'orders', COUNT(*) FROM taxi.orders;
```

Результат:

Таблица	Количество записей
position	3
tariff_price	3
car_model	5
client	5

bank_card	3
contract	5
car	5
tariff	6
employee	5
orders	5

Создание резервной копии БД:

Для резервного копирования использовалось средство Backup в pgAdmin 4.

Были созданы две резервные копии базы данных:

- taxi_service.custom
- taxi_service.sql

Восстановление баз данных:

Для проверки работоспособности резервной копии была создана новая база данных.

Восстановление выполнено средствами pgAdmin 4 через меню Restore.

После восстановления выполнена проверка наличия таблиц и записей.

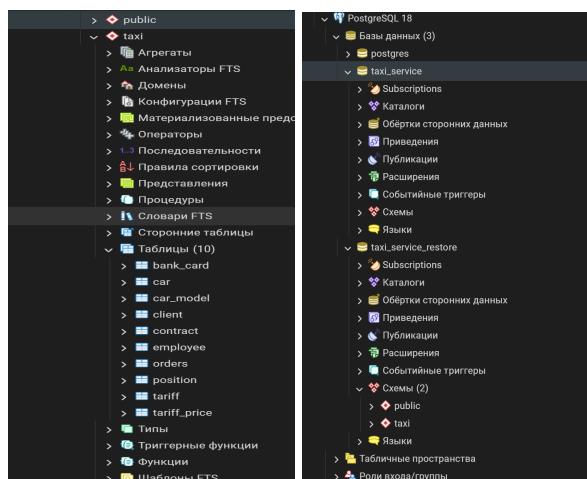
Проверочный запрос:

```
SELECT COUNT(*) FROM taxi.orders;
```

Результат:

```
5
```

Итоговая структура в PgAdmin 4:



Вывод:

В ходе выполнения лабораторной работы была создана база данных PostgreSQL для предметной области «Служба заказа такси».

Была создана схема taxi, разработаны таблицы базы данных, определены ограничения целостности Primary Key, Foreign Key, Unique и Check.

Таблицы были заполнены рабочими данными и проверены с помощью SQL-запросов.

Также были изучены средства резервного копирования и восстановления PostgreSQL. Созданы резервные копии базы данных в форматах Custom и SQL, а также выполнено успешное восстановление базы данных из резервной копии.

Получены практические навыки проектирования и администрирования баз данных PostgreSQL.